Dynamically Maintaining Conversation History

1. Today's Most Important Pattern

This is the core coding pattern for maintaining conversation history dynamically.

```
messages.append(
    HumanMessage(content=user_input)
)

response = llm.invoke(messages)

messages.append(response)
```

Core Flow

Add Human Message

↓

Send Complete History to Model

↓

Get AI Response

↓

Add AI Response to History

↓

Repeat

This cycle continues for every new user message.

---

2. Why Is This Pattern Important?

For every conversation turn:

User enters message

↓

HumanMessage is created

↓

Message is added to history

↓

Complete history goes to model

↓

Model generates AIMessage

↓

AIMessage is added to history

The updated history is then ready for the next turn.

---

3. Program 1 — First Dynamic Conversation History Program

Start with an empty list.

Then:

1. Add the human message.
2. Send the complete list to the model.
3. Receive the AI response.
4. Add the AI response back into the list.

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage



llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
# Empty conversation history
messages = []


# User message
user_input = "My name is Durga."


# Add user message to history
messages.append(
    HumanMessage(
        content=user_input
    )
)


# Send complete history to model
response = llm.invoke(
    messages
)
```

```
print("AI:", response.content)



# Add AI response to history
messages.append(
    response
)



print("\nConversation History:")
print(messages)
```

---

4. Program Flow

Initially:

```
messages = []
```

Then:

```
messages = []
     ↓
```

Add HumanMessage

↓

llm.invoke(messages)

↓

Receive AIMessage

↓

Add AIMessage

↓

Updated History

---

5. Very Important Point

The object:

response

returned by the chat model is already an:

AIMessage

Therefore, we can directly write:

messages.append(response)

We do not need to manually create another AIMessage.

---

6. History After One Conversation Turn

Suppose the user says:

My name is Durga.

After the AI replies, the list conceptually becomes:

messages

```
  |
  ├── HumanMessage("My name is Durga.")
  |
  └── AIMessage("Nice to meet you, Durga.")
```

Now both messages are available for the next model call.

---

7. Program 2 — Interactive Chatbot with Exit

Now we can repeat the same logic using a loop.

```
import os


from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage



llm = ChatOpenAI(
   model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
)


messages = []


while True:

    user_input = input(
        "You: "
    )

    # Stop chatbot
    if user_input.lower() == "exit":
        print("Chat ended.")
        break

    # Add user message
    messages.append(
```

```
    HumanMessage(
        content=user_input
    )
)

# Send complete history
response = llm.invoke(
    messages
)

# Display AI response
print(
    "AI:",
    response.content
)

# Add AI response
messages.append(
    response
)
```

```
print(
    "\nNumber of messages:",
    len(messages)
)
```

---

8. How the Chatbot Works

Every loop iteration performs:

Read User Input

↓

Append HumanMessage

↓

Send Complete messages List

↓

Get AI Response

↓

Print AI Response

↓

Append AIMessage

↓

Repeat

---

9. Why Do We Use a while Loop?

The loop allows continuous conversation.

Without a loop:

One Question

↓

One Response

↓

Program Ends

With a loop:

Question 1

↓

Answer 1

↓

Question 2

↓

Answer 2

↓

Question 3

↓

Answer 3

↓

...

---

10. How Does exit Work?

We check:

if user_input.lower() == "exit":

If the user enters:

exit

the program executes:

break

and stops the loop.

---

11. Why Do We Use user_input.lower()?

Suppose the user types:

EXIT

or:

Exit

or:

exit

After:

user_input.lower()

all become:

exit

So the chatbot can stop correctly.

---

12. Number of Messages

We can check:

len(messages)

After one complete conversation turn:

HumanMessage

AIMessage

Number of messages:

2

After two complete turns:

HumanMessage

AIMessage

HumanMessage

AIMessage

Number of messages:

4

So normally:

1 complete turn

↓

2 messages

---

13. Program 3 — Print Complete Conversation History

We can also display the complete conversation history.

```
import os

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage



llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
messages = []


while True:

    user_input = input(
        "\nYou: "
    )

    if user_input.lower() == "exit":
        print("Chat ended.")
        break

    messages.append(
        HumanMessage(
            content=user_input
        )
    )
```

```
response = llm.invoke(
    messages
)

print(
    "AI:",
    response.content
)

messages.append(
    response
)



# ------------------------------
# PRINT CURRENT HISTORY
# ------------------------------

print("\nCurrent History:")
print("----------------")
```

```
for message in messages:
    print(
        type(message).__name__,
        ":",
        message.content
    )
```

---

14. Understanding the History Loop

This part:

for message in messages:

visits every message stored in the list.

Then:

type(message).__name__

tells us the message type.

For example:

HumanMessage

AIMessage

And:

message.content

gives the actual message text.

---

15. Example History Output

Suppose conversation is:

You: My name is Durga.

AI: Nice to meet you, Durga.

You: What is my name?

AI: Your name is Durga.

The history may look like:

Current History:

HumanMessage : My name is Durga.

AIMessage : Nice to meet you, Durga.

HumanMessage : What is my name?

AIMessage : Your name is Durga.

---

16. Memory-Like Behaviour

Now we can see memory-like behaviour being created through the Python list.

messages

↓

Stores previous conversation

↓

Sent again to model

↓

Model gets previous context

↓

Memory-like behaviour

The important point is:

The application maintains history.

---

17. Why Learn Manual History First?

Later, we may use something like:

RunnableWithMessageHistory

But before learning that, we should clearly understand the basic problem it solves.

Without understanding:

messages.append(...)

we may feel that conversation memory is some kind of magic.

But now we know that history must be:

Stored

↓

Updated

↓

Retrieved

↓

Passed to Model

---

18. Core Manual History Logic

HISTORY

│

├── Store

│

├── Update

│

├── Retrieve

│

└── Pass to Model

This is the foundation for more advanced conversation-history systems.

---

19. Limitation 1 — In-Memory History Only

Our current program contains:

messages = []

This is only a Python list in the running program.

What happens when we stop the Python program?

Program Stops

↓

Python Process Ends

↓

messages List Disappears

If we run the program again:

messages = []

starts empty again.

---

20. In-Memory History

Our current history is:

In-Memory History

Meaning:

Available

only during

current program execution

Persistent history is a later topic.

---

21. Limitation 2 — No Separate Conversation History

Our program currently has:

messages = []

This represents only one conversation history.

Suppose:

Durga is chatting

Ravi is chatting

Suresh is chatting

Should all three use the same history list?

No

Each user should have separate conversation history.

---

22. Separate User Histories

Conceptually:

Durga

↓

History 1

Ravi

↓

History 2

Suresh

↓

History 3

This leads us later to concepts like:

Session ID

Thread ID

Separate History per Session

---

23. Why Is Separate History Required?

Suppose Durga says:

My favourite language is Python.

And Ravi says:

My favourite language is Java.

If both users share one history:

Durga History

+

Ravi History

the conversations may get mixed.

So every conversation needs proper identification.

---

24. Limitation 3 — Growing History Problem

Our list keeps growing.

Example:

2 messages

4 messages

10 messages

50 messages

100 messages

...

Every new user message and AI response is added to the list.

---

25. Why Is Growing History a Problem?

Growing conversation history can affect:

Context Window

+

Token Usage

+

Cost

+

Latency

---

26. Growing History Flow

More Conversation

↓

More Messages

↓

More Tokens

↓

More Context Usage

↓

Potentially More Cost

↓

Potentially More Latency

This is another reason why conversation-history management becomes important.

---

27. Current Manual Approach — Limitations Summary

Limitation	Meaning
In-memory only	History disappears when program stops
One history list	No separate user/session history
Growing history	More messages can increase context and token usage

---

28. Common Mistake 1 — Wrong Order

Wrong:

```
response = llm.invoke(messages)


messages.append(
   HumanMessage(
      content=user_input
```

```
    )
)
```

Here the latest user question is added **after** model invocation.

So the model does not receive the latest question in that call.

---

29. Correct Order

Correct:

```
messages.append(
    HumanMessage(
        content=user_input
    )
)

response = llm.invoke(
    messages
)
```

Memory Rule

First Append Human

↓

Then Invoke Model

---

30. Common Mistake 2 — Forgetting AI Response

If we forget:

messages.append(response)

then the AI responses are missing from future history.

Correct flow:

Append Human

↓

Invoke Model

↓

Append AI

---

31. Why Must We Store AI Responses?

Conversation history should normally contain both sides.

Human Message

+

AI Message

If we store only human messages:

Human

Human

Human

the model does not receive the complete previous dialogue.

---

32. Common Mistake 3 — Creating messages = [] Inside the Loop

Wrong:

```
while True:

    messages = []
```

Ayyooo!

Every iteration creates a new empty list.

That means previous conversation history is lost.

---

33. Correct Placement

Correct:

```
messages = []

while True:
    ...
```

Create the history list **before the loop**.

Very very important.

---

34. Why Before the Loop?

Wrong

Loop Starts

↓

New Empty List

↓

Conversation

↓

Next Iteration

↓

New Empty List Again

Previous history is lost.

Correct

Create One List

↓

Start Loop

↓

Keep Updating Same List

↓

History Continues

---

35. Common Mistake 4 — Sending Only user_input

Wrong for our conversation-history example:

llm.invoke(user_input)

This sends only the latest input.

It does not send our maintained history list.

---

36. Correct Call

For conversation history:

llm.invoke(messages)

is the important point.

Difference

llm.invoke(user_input)

↓

Latest input only

llm.invoke(messages)

↓

Complete maintained history

---

37. Common Mistake 5 — Appending Only Strings

Avoid:

```
messages.append(
    user_input
)
```

For our message-based conversation example, use:

```
messages.append(
    HumanMessage(
        content=user_input
    )
)
```

This makes the conversation role clear.

---

38. Why Message Objects?

Using:

```
HumanMessage(...)
```

clearly tells the chat model:

This message came from the human.

Similarly, the returned response is an:

AIMessage

So our history has clear roles.

---

39. Correct Dynamic Conversation Pattern

```
messages = []

while True:

    user_input = input("You: ")

    messages.append(
        HumanMessage(
            content=user_input
        )
    )

    response = llm.invoke(
        messages
    )
```

```
print(
    "AI:",
    response.content
)

messages.append(
    response
)
```

This is the core concept of today's lesson.

---

40. Master Flow

```
START
  ↓
messages = []
  ↓
Read User Input
  ↓
Create HumanMessage
  ↓
Append HumanMessage
```

↓

Send Complete History

↓

llm.invoke()

↓

Receive AIMessage

↓

Display AI Response

↓

Append AIMessage

↓

Updated History

↓

Next User Input

↓

REPEAT

---

41. Important Key Points

1. Conversation history can be maintained using a Python list.

2. User input is added as a HumanMessage.
3. The complete messages list is passed to the model.
4. Model response is already an AIMessage, so we can append it directly.
5. A loop converts the same logic into a multi-turn chatbot.
6. The messages list should normally be created before the loop.
7. Both human and AI messages should be preserved.
8. Manual history gives memory-like conversation behaviour.
9. Current list-based history exists only while the program is running.
10. Real applications need separate history for different sessions or users.
11. Conversation history can grow and affect context window, token usage, cost, and latency.

---

42. MCQ 1

Where should we create this list?

messages = []

A. Inside every loop iteration
B. Before the chat loop

C. After the program ends
D. Inside response.content

Answer: B

Conclusion

Create the history once before the loop so previous messages remain available.

---

43. MCQ 2

Which line sends previous conversation history to the model?

A.

llm.invoke(user_input)

B.

llm.invoke(messages)

C.

print(messages)

D.

messages.clear()

Answer: B

Conclusion

llm.invoke(messages) sends the complete currently maintained message history.

---

44. MCQ 3

Why do we execute:

messages.append(response)

A. To retrain the model
B. To store the AI response in conversation history
C. To create an API key
D. To clear conversation history

Answer: B

Conclusion

The AI response becomes part of the history for future conversation turns.

---

45. Final Revision Diagram

CONVERSATION HISTORY

messages

|

↓

HumanMessage added

|

↓

llm.invoke(messages)

|

↓

AIMessage

|

↓

AIMessage added

|

↓

Updated messages

|

↓

Next Turn

---

46. Easy Memory Formula

HUMAN

↓

APPEND

↓

INVOKE

↓

AI

↓

APPEND

↓

REPEAT

Or simply remember:

Append Human

↓

Invoke Model

↓

Append AI

↓

Repeat

---

47. Manual Conversation History

messages = []

↓

History exists in application

↓

History keeps growing

↓

Complete history sent to model

↓

Model gets conversation context

---

48. Present Approach vs Later Approach

Current Approach

Manual History

↓

messages = []

↓

messages.append(...)

Later

We may use higher-level mechanisms such as:

RunnableWithMessageHistory

But the underlying problem remains the same:

Store History

↓

Update History

↓

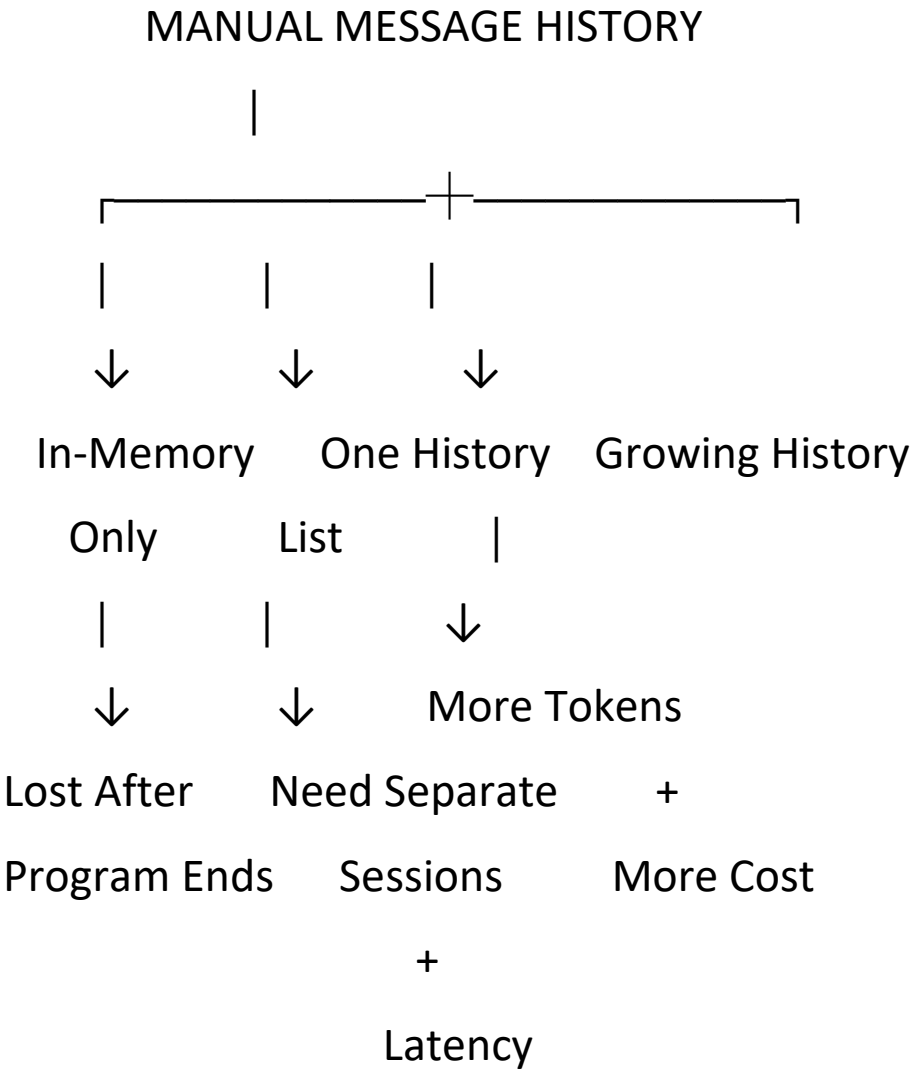
Retrieve History

↓

Provide History to Model

---

49. Master Limitations Diagram



---

50. One-Line Conclusion

A conversation chatbot works by continuously adding the user's message and the AI's response to a history and sending that updated history back to the model.

Final Memory Line

User Message

↓

HumanMessage

↓

Append to History

↓

Send History to Model

↓

AIMessage

↓

Append to History

↓

Repeat